



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

软件构造

第十章：多线程



多线程

- 进程与线程
- 多线程
- Java中对线程的控制
- 同步、死锁及如何避免
- 任务与线程池
- 多线程应用——生产者与消费者模式



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

进程与线程



什么是进程

❑ 进程（Process）：正在运行的程序的**实例**

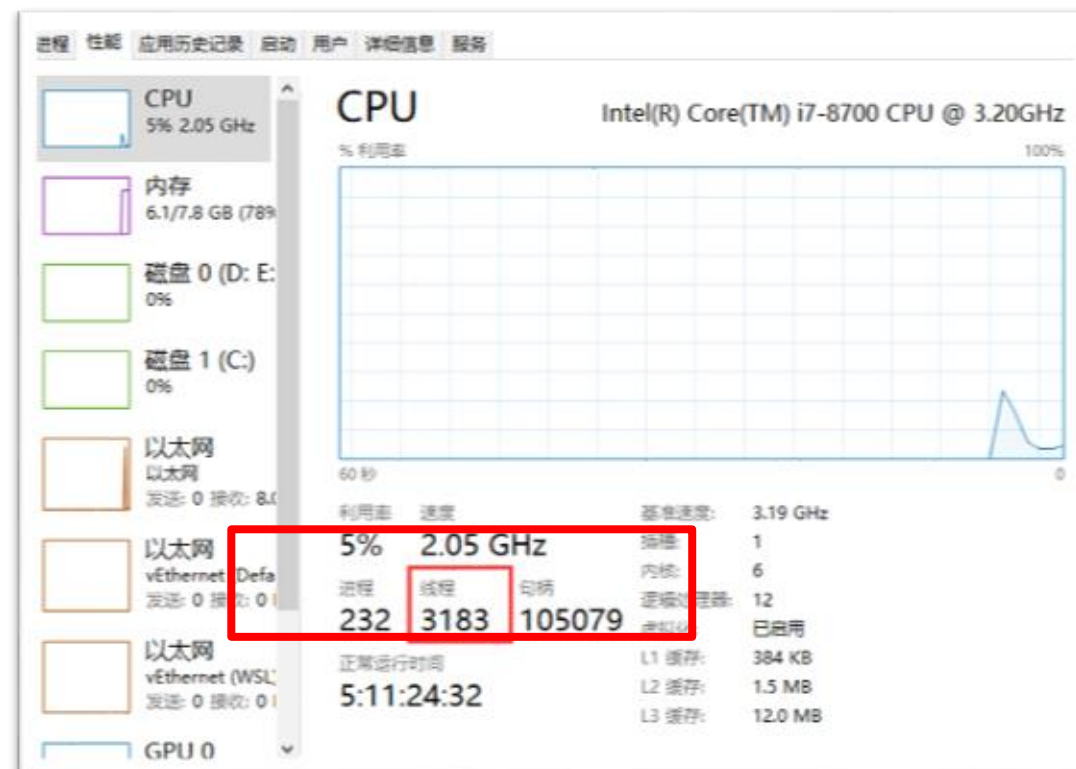
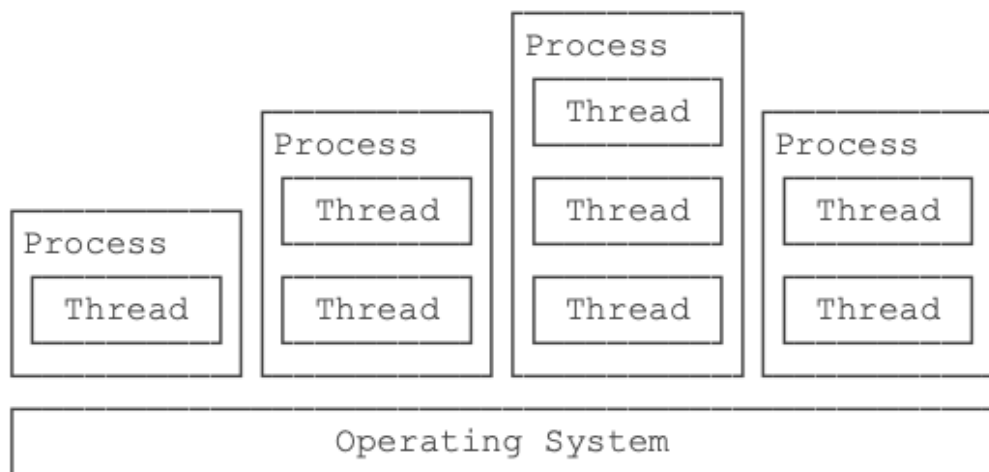
- **私有空间**，彼此隔离
- 多进程之间**不共享内存**
- 是操作系统**分配资源的基本单位**
- 进程之间通过消息传递进行协作
- 一般来说：进程 == 程序 == 应用
 - ✓ 但一个应用中也可能包含多个进程

名称	状态	2% CPU	35% 内存	1% 磁盘
应用 (9)				
> Google Chrome (23)		0.3%	1,266.5 ...	0 MB/秒
> Microsoft Edge (19)		0%	682.3 MB	0 MB/秒
> Microsoft Excel		0%	38.7 MB	0 MB/秒
▼ Microsoft PowerPoint (2)		0%	408.0 MB	0 MB/秒
P 第九章-Swing介绍V6 - P...				
P 第十章: 多线程v7 - Pow...				
> Microsoft Word (2)		0%	170.1 MB	0 MB/秒
> RainClassroom5.0 (32 位) (6)		0%	115.8 MB	0.1 MB/秒
> WeChat (32 位) (13)		0.9%	478.2 MB	0.1 MB/秒
> Windows 资源管理器		0%	122.4 MB	0 MB/秒
> 任务管理器		0.1%	71.9 MB	0 MB/秒
后台进程 (97)				
360安全浏览器 服务组件 (32 ...		0%	3.3 MB	0 MB/秒

什么是线程

❑ 线程（Thread）：进程中一个单一顺序的控制流

- 操作系统能够进行运算调度的最小单位，是CPU调度的基本单位
- 包含在进程中，是进程的实际运作单位
- 一个进程可以包含多个线程
- 一个进程至少包含一个线程
- 多个线程之间共享内存





线程与进程

进程（Process）	线程（Thread）
重量级	轻量级
一个应用可以包含多个进程	一个进程可以包含多个线程
多个进程间 不共享内存	一个进程的多个线程间 共享内存
进程表现为 虚拟机	线程表现为 虚拟CPU

资源分配的最小单位

程序执行的最小单位



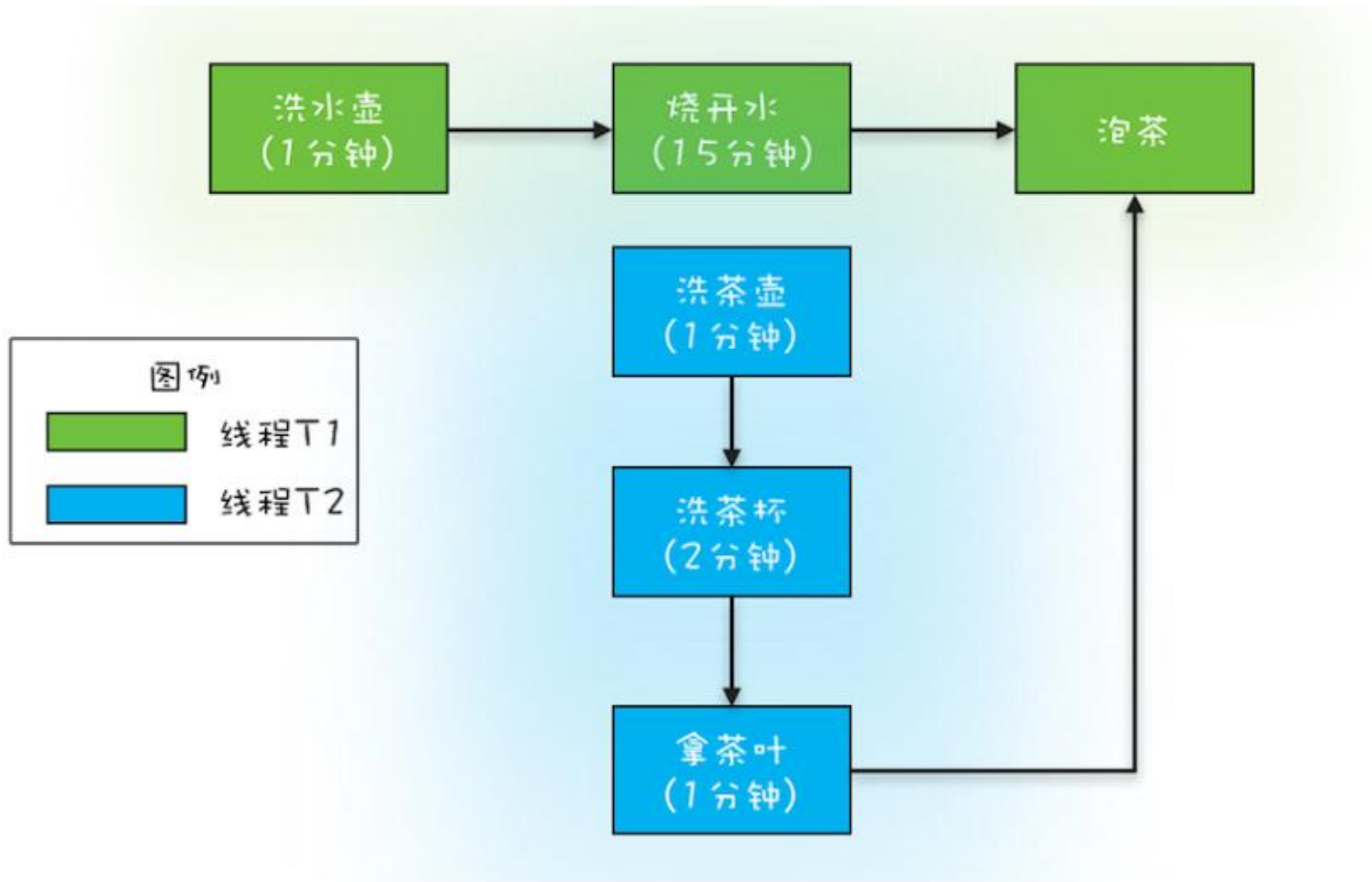
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

多线程



生活中的“多线程”



烧水泡茶最优分工方案



飞机大战中的多线程

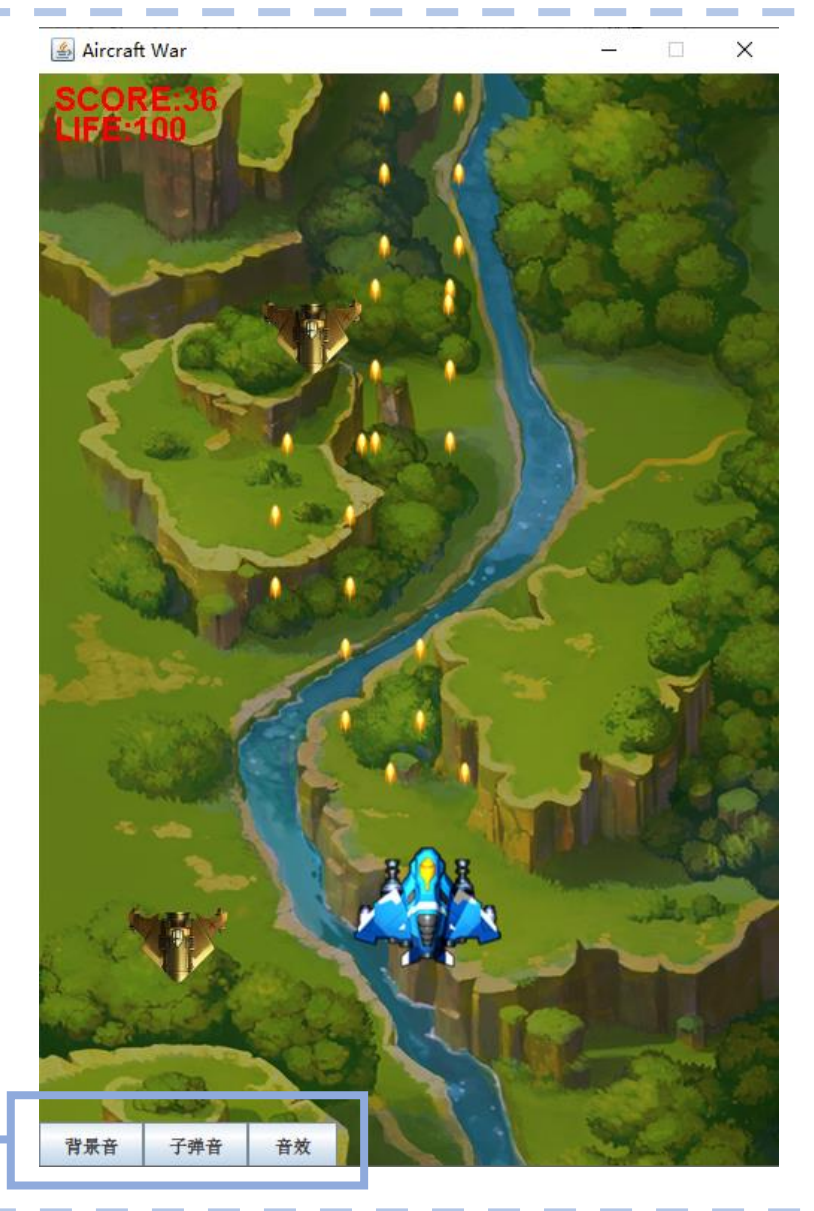
飞机大战应用 进程

游戏控制主线程

游戏逻辑线程

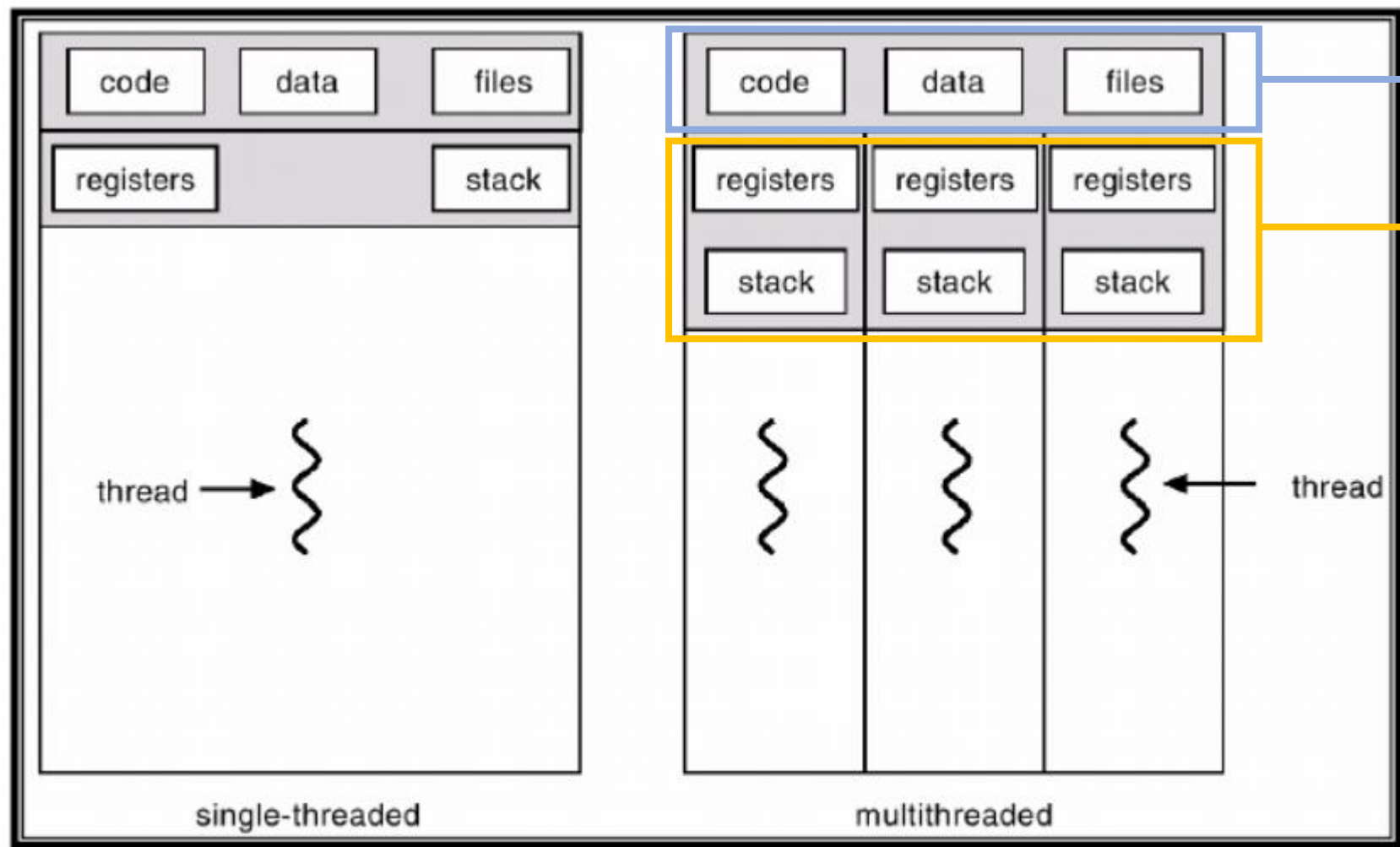
音效控制线程

鼠标/键盘监听线程





多线程



共享内存（共享同一个地址空间）

不共享寄存器与栈



互动小问题

堆是线程共享的吗？

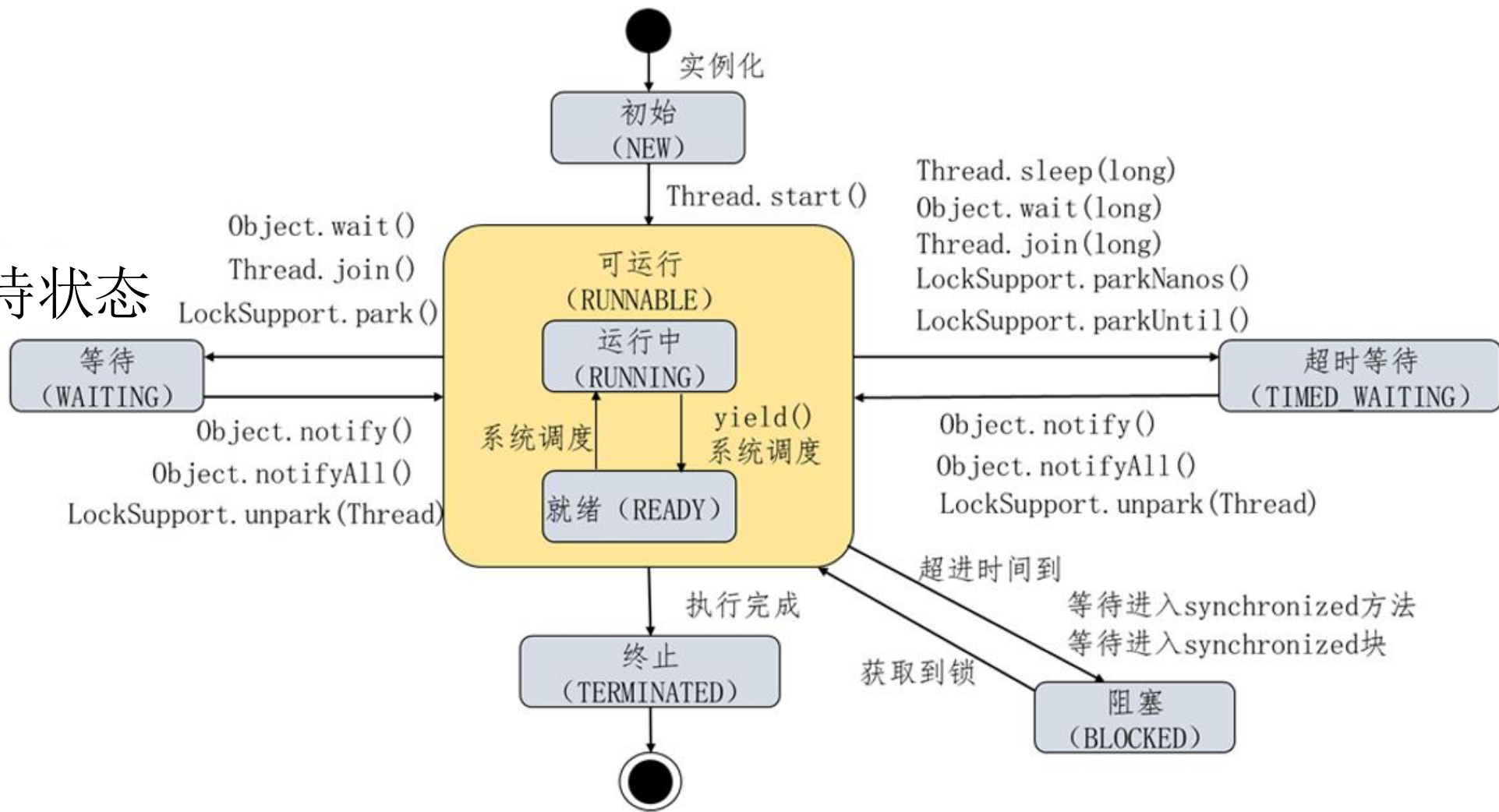


Java中对线程的控制



线程的状态

- 初始状态
- 可运行状态
- 阻塞状态
- 等待状态
- 计时/超时等待状态
- 终止状态





创建线程

• 两种方式

➤ 继承 Thread 类

➤ 实现 Runnable 接口

• 调用 start 方法

➤ 启动线程，将引发调用 run 方法；

➤ start 方法将立即返回；

➤ 新线程和原线程将并发运行。

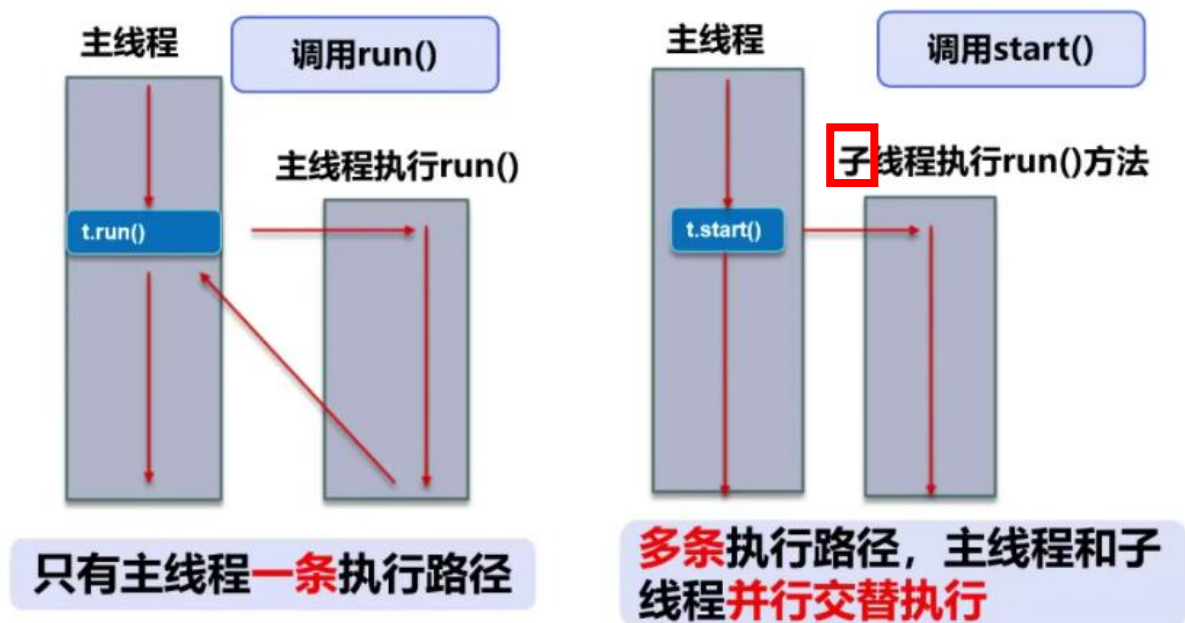
• 注意：不能直接调用 run 方法

➤ 直接调用 run 方法只会执行同一个线程中的任务，而不会启动新线程。

```
1 public class Thread1 extends Thread {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}
```

```
2 public class Thread2 implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args){  
        new Thread1().start();  
        new Thread(new Thread2()).start();  
    }  
}
```





创建线程

- Runnable更常用，其优势在于：

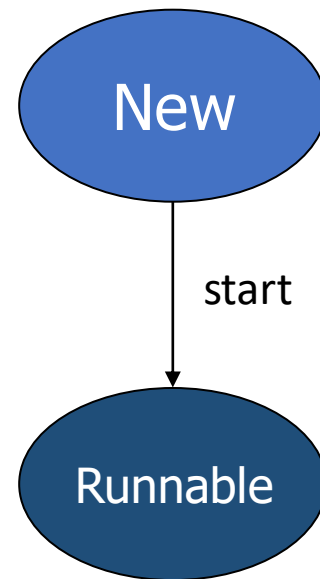
- 避免由于单继承局限所带来的影响；
- 任务与运行机制解耦，降低开销；
- 更容易实现多线程资源共享。

- Java8后引入的lambda语法进一步简写为：

- 创建并启动线程

```
public static void main(String[] args) {  
    //lambda 表达式  
    Runnable r = () -> {  
        System.out.println("Thread: Runnable with Lambda Expression");  
    };  
    // 启动线程  
    new Thread(r).start();  
}
```

```
public class Thread1 extends Thread {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
public class Thread2 implements Runnable {  
    @Override  
    public void run() {  
        System.out.println("New Thread");  
    }  
}  
  
public class Main {  
    public static void main(String[] args){  
        new Thread1().start();  
        new Thread(new Thread2()).start();  
    }  
}
```



互动小问题

在刚才买票的例子中(TicketDemo1.java), 怎么做到多线程共享资源?

互动小问题

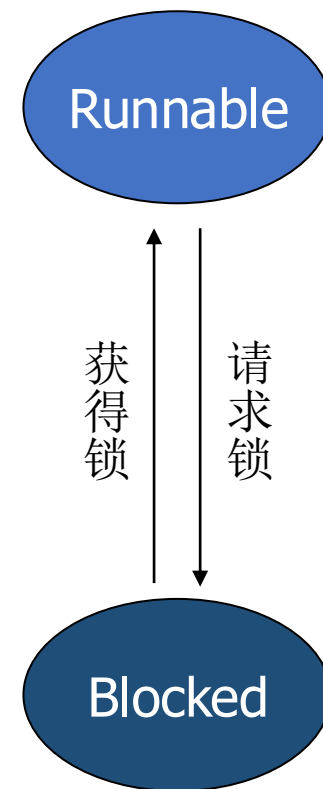
静态变量是线程安全的吗？

实例变量是线程安全的吗？

局部变量是线程安全的吗？

线程阻塞（被动进入）

- 线程何时进入阻塞状态：
 - 当一个线程试图获取一个内部的对象锁，而该锁被其他线程持有。
- 线程何时变成非阻塞状态：
 - 当所有其他线程释放该锁，且线程调度器允许本线程持有它的时候。
- 当线程处于被阻塞或等待状态时，它暂时不活动
 - 它不运行任何代码且消耗最少的资源。直到线程调度器重新激活它。





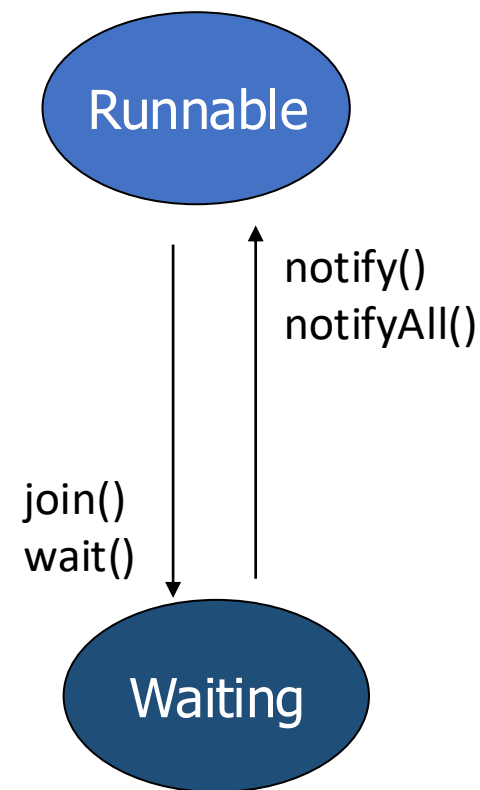
线程等待

- 运行→等待:

- 情形1: **当前**线程对象调用 **Object.wait()** 方法 (**主动等**) ;
- 情形2: **其他**线程调用 **Thread.join()** 方法 (**被动等**) 。

- 等待→运行:

- 情形1: 等待的线程被其他线程对象唤醒, 调用 **Object.notify()** 或者 **Object.notifyAll()**。
- 情形2: 调用 **Thread.join()** 方法的线程结束。



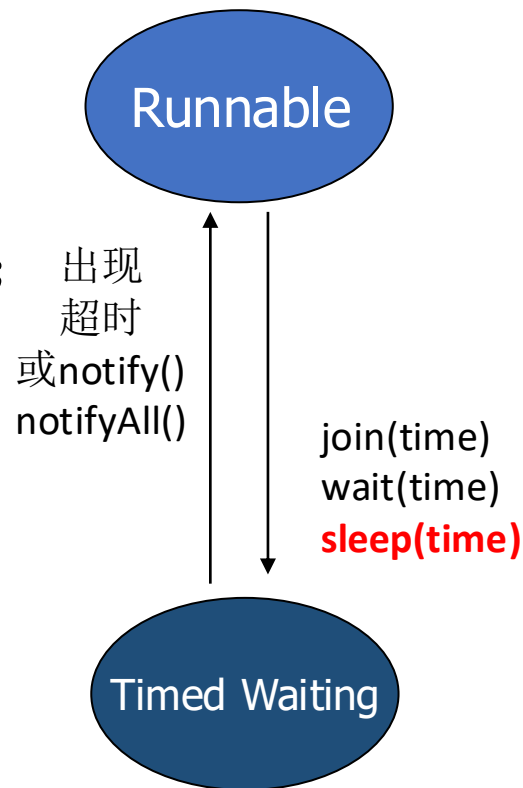


线程计时/超时等待

- 运行→计时等待:

- 情形1: 当前线程对象调用 **Object.wait(time)** 方法 (主动等) ;
- 情形2: 其他线程调用 **Thread.join(time)** 方法 (被动等) 。
- 情形3: 当前线程调用 **Thread.sleep(time)** 方法 (主动等) 。

sleep 方法是让当前线程休眠, 让出cpu, 但是**不释放锁**, 这是**Thread**的静态方法
wait 方法是让当前线程等待, **释放锁**, 这是**Object**的方法



- 计时等待→运行:

- 区别于等待, 它可以在**指定的时间**自行返回。

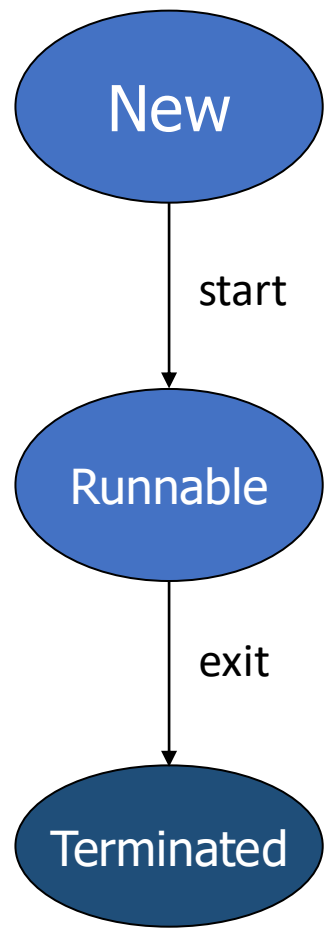


互动小问题

- sleep和wait方法的异同点？
 - 相同点：
 - ✓ 都可以让线程处于等待状态。
 - 不同点：
 - ✓ sleep方法**必须**指定时间；wait方法**可以**指定时间，**也可以**不指定。
 - ✓ sleep方法时间到，线程处于临时阻塞或者运行；wait如果没有指定时间，必须要通过**notify**或者 **notifyAll()**唤醒。
 - ✓ sleep方法是让当前线程休眠，让出cpu，但是**不释放锁**，这是**Thread**的静态方法；wait方法是让当前线程等待，**释放锁**，这是**Object**的方法。

终止线程

- 线程会由于以下两个原因之一而终止：
 - run 方法正常退出，线程自然终止；
 - 因为一个没有捕获的异常终止了 run 方法，使线程意外终止。
- 注意：终止线程不可以用stop 方法，此方法已被废弃
 - 该方法抛出 ThreadDeath 错误对象，由此杀死线程。
 - 不安全：该方法试图终止一个给定线程，但是是强行终止。
- 正确的退出方法：
 - 使用interrupt()方法中断线程。





中断线程

- 当对一个线程调用 `interrupt` 方法时，线程的**中断状态将被置位**
 - 这是每一个线程都具有的 `boolean` 标志（`true`：打断了；`false`：没有打断）。
 - 每个线程都应该不时地检查此标志（`isInterrupted()`来获取打断标志），**以判断线程是否被中断**。
- 当在一个被阻塞的线程上调用 `interrupt` 方法
 - 阻塞调用（`sleep`,`wait`,`join`调用）将会被`InterruptedException` 异常中断。
- 中断一个线程**不过是引起它的注意**
 - 被中断的线程可以决定如何响应中断。



线程优先级

- 每一个线程都有一个**优先级**
 - 默认情况下，一个线程继承它的父线程的优先级；
 - 可以用 `setPriority(int newPriority)` (1~10, 默认值5) 方法提高或降低任何一个线程的优先级。
- 每当调度器决定运行一个新线程时，首先会在**具有高优先级的线程中进行选择**，但只是**提高**优先级高的线程先执行的**概率**。
 - 优先级高的线程不代表一定比优先级低的线程优先执行。
- 线程优先级是**高度依赖于系统的**
 - 在 Oracle 为 Linux 提供的 Java 虚拟机中，线程的优先级被忽略。



守护线程

- 使用 `setDaemon` 方法标识该线程为守护线程或用户线程。
 - `thread.setDaemon(true);` // true:守护线程
- 守护线程的唯一用途是**为其他线程提供服务**。
 - 例子：计时线程。它定时地发送“计时器嘀嗒”信号给其他线程。
 - 最典型的守护线程：GC（**垃圾回收器**），它就是一个很称职的守护者
- 在JVM中，所有非守护线程都执行完毕后，无论有没有守护线程，虚拟机都会**自动**退出。
 - **注意**：由于守护线程的终止是自身无法控制的，因此千万不要把IO、File等重要操作逻辑分配给它。



互动小问题

```
public class DaemonTest {  
    public static void main(String[] args){  
        Thread thread = new Thread(new Runnable() {  
            @Override  
            public void run() {  
                try {  
                    Thread.sleep(1000);  
                } catch (InterruptedException e) {  
                    e.printStackTrace();  
                }  
                int sum = 0;  
                for (int i = 1; i <= 100; i++) {  
                    sum = sum + i; }  
                System.out.println("守护线程，最终求和的值为：" + sum);  
            }  
        });  
        thread.setDaemon(true); //设置thread为守护线程  
        thread.start();  
        System.out.println("main 函数线程执行完毕，JVM 退出。");  
    }  
}
```

结果是什么？

线程的相关方法总结（选看）

主要总结Thread类中的核心方法

方法名称	是否static	方法说明
start()	否	让线程启动，进入就绪状态,等待cpu分配时间片
run()	否	重写Runnable接口的方法,线程获取到cpu时间片时执行的具体逻辑
yield()	是	线程的礼让，使得获取到cpu时间片的线程进入就绪状态，重新争抢时间片
sleep(time)	是	线程休眠固定时间，进入阻塞状态，休眠时间完成后重新争抢时间片,休眠可被打断
join()/join(time)	否	调用线程对象的join方法，调用者线程进入阻塞,等待线程对象执行完或者到达指定时间才恢复，重新争抢时间片
isInterrupted()	否	获取线程的打断标记，true:被打断，false：没有被打断。调用后不会修改打断标记
interrupt()	否	打断线程，抛出InterruptedException异常的方法均可被打断，但是打断后不会修改打断标记，正常执行的线程被打断后会修改打断标记
interrupted()	否	获取线程的打断标记。调用后会清空打断标记
currentThread()	是	获取当前线程

线程的相关方法总结（选看）

Object中与线程相关方法

方法名称	方法说明
wait()/wait(long timeout)	获取到锁的线程进入阻塞状态
notify()	随机唤醒被wait()的一个线程
notifyAll();	唤醒被wait()的所有线程，重新争抢时间片



同步、死锁及如何避免



线程同步

- 当多个线程同时运行时，**线程的调度由操作系统决定**，程序本身无法决定。因此，任何一个线程都有可能在任何指令处被操作系统暂停，然后在某个时间段后继续执行。
- 如果多个线程**同时读写共享变量**，会出现数据不一致的问题。

```
class Counter {  
    public static int count = 0;  
}
```

```
class AddThread extends Thread {  
    public void run() {  
        for (int i=0; i<100000; i++) { Counter.count += 1;  
        }  
    }  
}
```

```
class DecThread extends Thread {  
    public void run() {  
        for (int i=0; i<100000; i++) { Counter.count -= 1;  
        }  
    }  
}
```



线程同步

- **原子操作**：指不能被中断的一个或一系列操作。即要么**完全执行**，要么**完全不执行**，不存在执行了一半的情况。

- 例子： $n = n + 1$ ；看上去是一行语句，实际上对应了**3条**指令

ILOAD

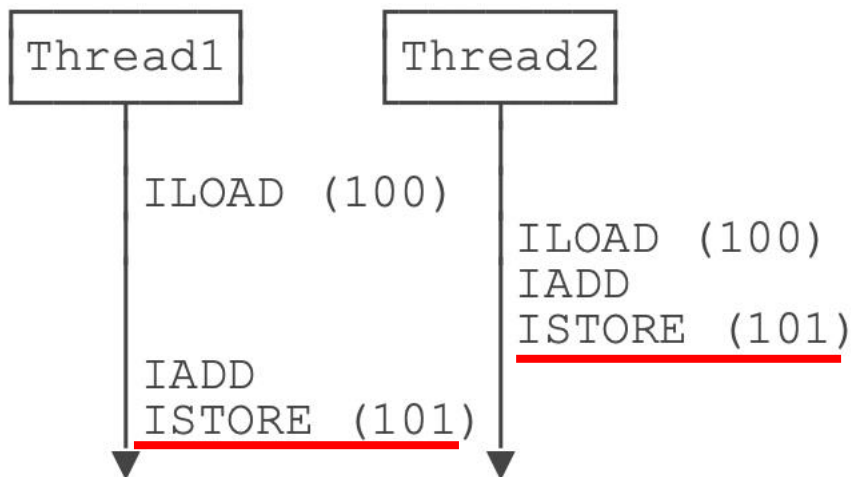
取数

IADD

加法运算

ISTORE

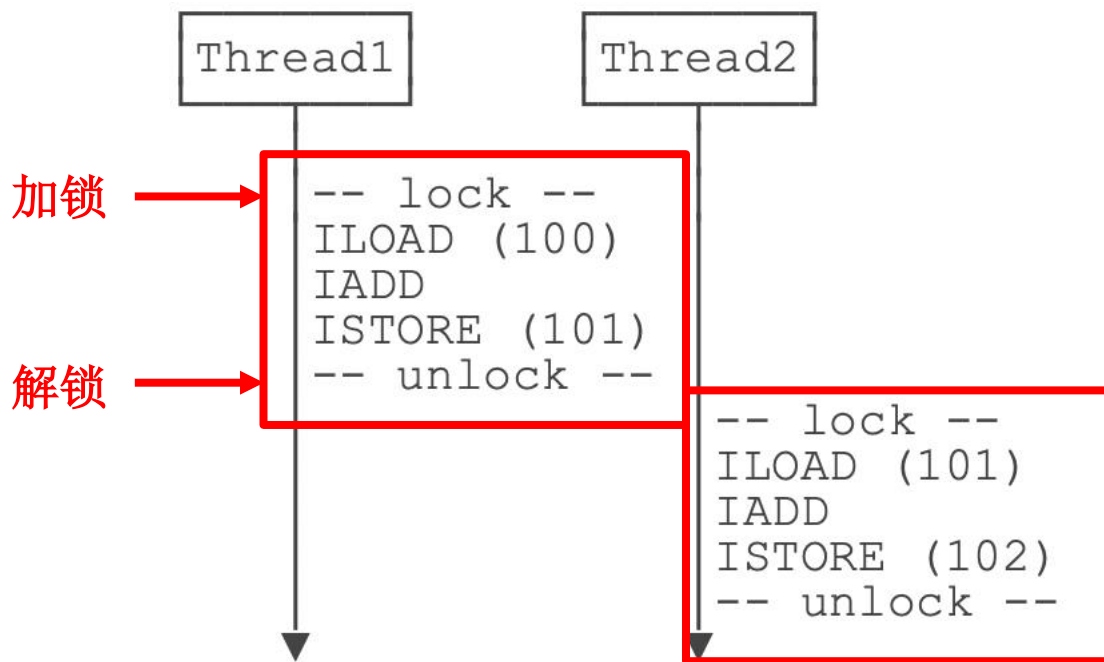
结果保存





线程同步

- 这说明多线程模型下，要**保证逻辑正确**，对共享变量进行读写时，必须保证一组指令以**原子方式**执行：即某一个线程执行时，其他线程必须等待。
- 通过**加锁和解锁**的操作，保证**3条指令总是在一个线程执行期间**，不会有其他线程会进入此指令区间。
- 即使在执行期线程被操作系统中断执行，其他线程也会因为**无法获得锁**导致无法进入此指令区间。





线程同步

- 代码实现:

- Java程序使用synchronized关键字对一个对象进行加锁。

```
synchronized(lock) {  
    n = n + 1;  
}
```

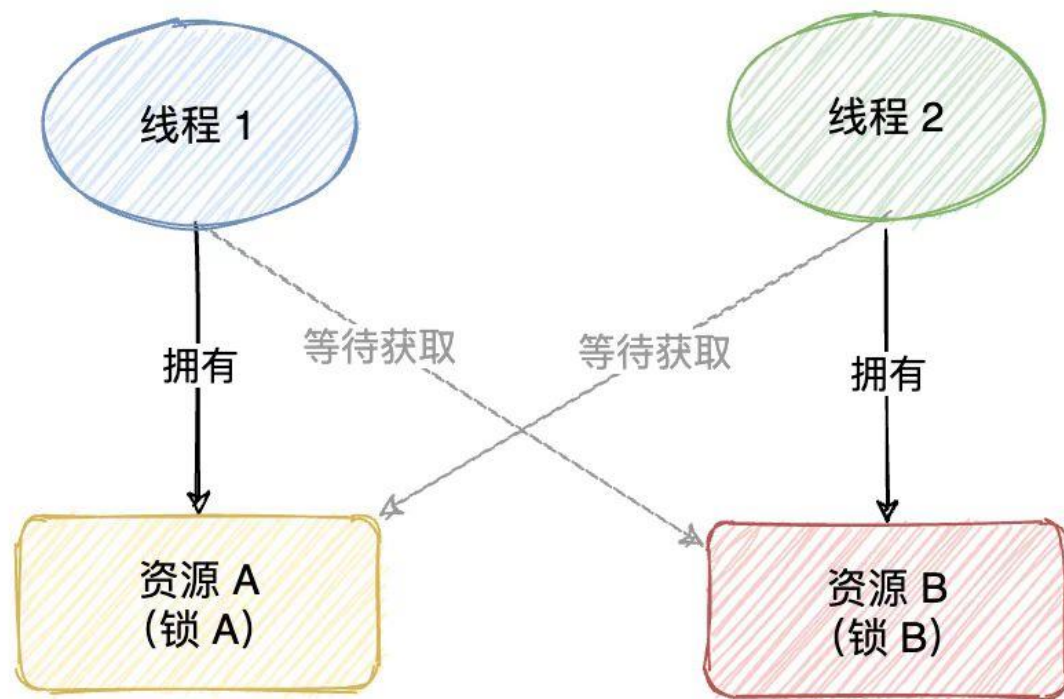
- 改写之前的代码

```
class AddThread extends Thread {  
    public void run() {  
        for (int i=0; i<10000; i++) {  
            synchronized(Counter.lock) { // 获取锁  
                Counter.count += 1;  
            } // 释放锁  
        }  
    }  
}
```

```
class DecThread extends Thread {  
    public void run() {  
        for (int i=0; i<10000; i++) {  
            synchronized(Counter.lock) { // 获取锁  
                Counter.count -= 1;  
            } // 释放锁  
        }  
    }  
}
```



线程死锁



- **死锁**:在线程A持有饭并想获得钱的同时，线程B持有钱并尝试获得饭，那么这两个线程将永远地等待下去。
- **避免死锁**:线程获取锁的**顺序要一致**。即严格按照先获取**钱**，再获取**饭**的顺序，或者先获取**饭**，再获取**钱**的顺序。



线程死锁

```
class PerA implements Runnable{
    public void run() {
        try {
            System.out.println( " PerA: 我有钱请帮我带饭");
            while(true){
                synchronized (DeadLock.Money) {
                    System.out.println(" PerA 锁住钱");
                    Thread.sleep(3000); // 此处等待是给PerB机会
                }
                synchronized (DeadLock.Food) {
                    System.out.println(" PerA 吃到饭");
                    Thread.sleep(60 * 1000); // 为测试
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
class PerB implements Runnable{
    public void run() {
        try {
            System.out.println(" PerB: 我有饭请先给我钱");
            while(true){
                synchronized (DeadLock.Food) {
                    System.out.println(" PerB 锁住饭");
                    Thread.sleep(3000); // 此处等待是给PerA机会
                }
                synchronized (DeadLock.Money) {
                    System.out.println(" PerB拿到钱");
                    Thread.sleep(60 * 1000); // 为测试
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



任务与线程池

Callable接口

问题：之前提到的Runnable接口的run方法没有参数和返回值，在运算结束后不能直接返回一个结果。但我们经常会遇到这样一个问题，就是希望线程结束时返回一个运算结果，那怎么办？

- Runnable：封装一个异步运行的任务，没有参数和返回值
- Callable：封装一个异步运行的任务，有返回值
 - 需实现 **call** 方法
 - Callable<Integer>表示返回Integer对象的异步计算任务
 - 可以抛出异常
 - **不能**直接传入Thread进行线程操作



互动小问题

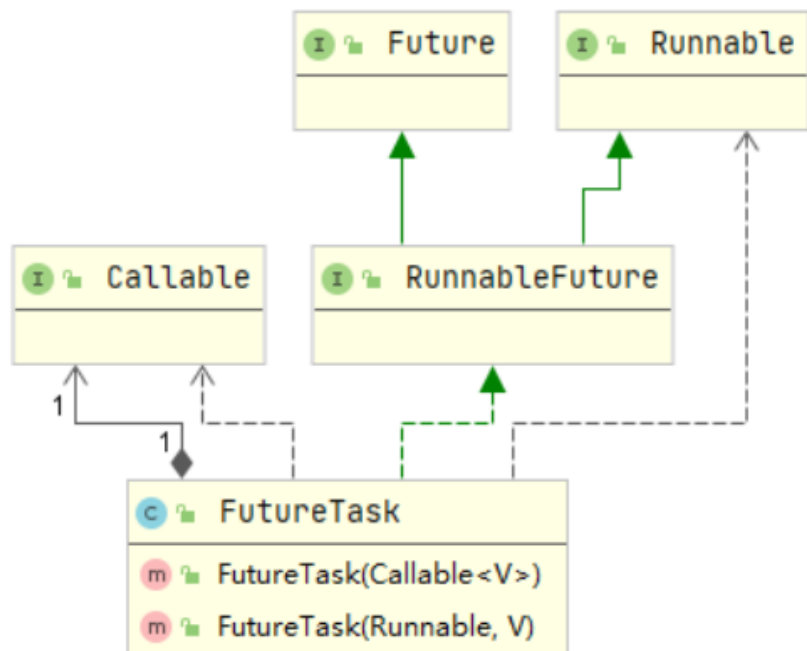
- Runnable和Callable的**区别**是什么？
 - ✓ Runnable执行方法是**run()**, Callable是**call()**。
 - ✓ Runnable**无返回值且不会抛异常**； Callable**有返回值且可以向外抛异常**。
 - ✓ Runnable实现类创建的对象**可以直接**传入Thread启动线程； Callable实现类创建的对象**不可以直接**传入Thread启动线程。

FutureTask

- FutureTask: 执行 Callable 的一种方法

- 控制 Callable 任务的执行，获得其运算结果

- 间接实现了 Runnable 接口，
进而可以通过 Thread 启动线程



```
// 实例化 Callable 任务，指定返回类型为 String
Callable<String> callable = new Callable<String>() {
    public String call() throws Exception {
        // balabalabala~
        return "Hello world ~";
    }
};

// 将 callable 任务委派给 FutureTask
FutureTask<String> task = new FutureTask<String>(callable);

// FutureTask实现了Runnable接口，所以可以直接传给 Thread 启动线程
new Thread(task).start();

// 通过 FutureTask 获取返回值
String call = task.get();

System.out.println(call); // print: Hello world ~
```

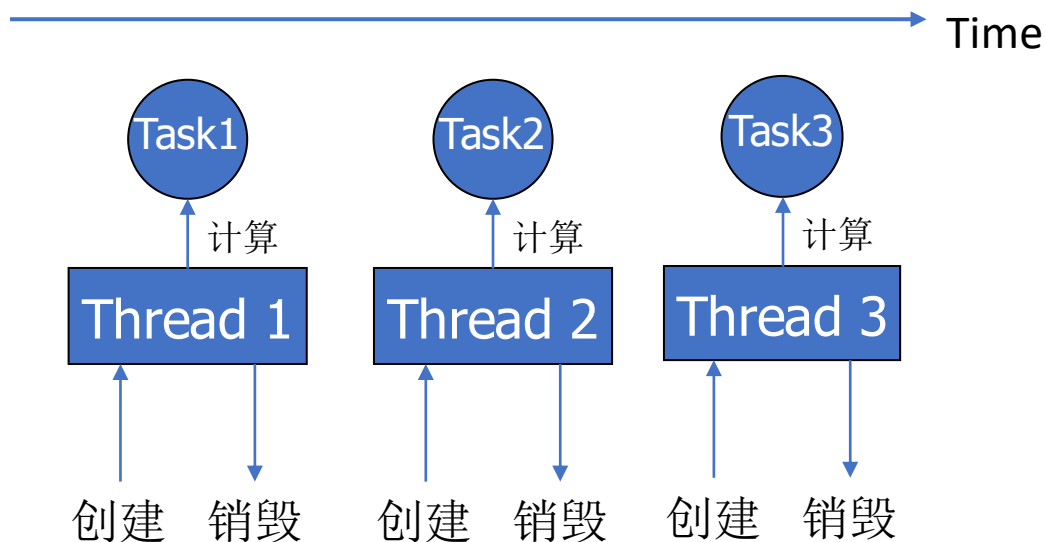
线程池

问题：频繁创建/终止大量线程，会带来大量开销，怎么办？

- 线程池（Thread Pool）：利用池化技术去减少资源对象的创建次数，提高程序的性能，特别是在高并发下这种提高更加明显。

是对资源的充分利用！

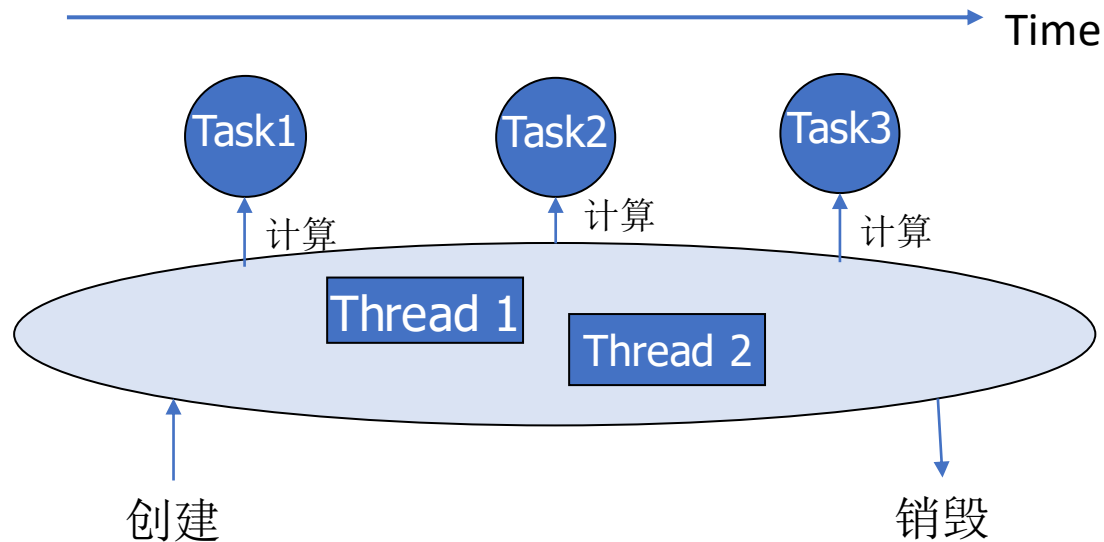
不使用线程池



为每个任务创建一个线程，任务完成后线程销毁

类比：每次有活时，招募工人，干完活遣散工人
再有活时，再招募工人

使用线程池

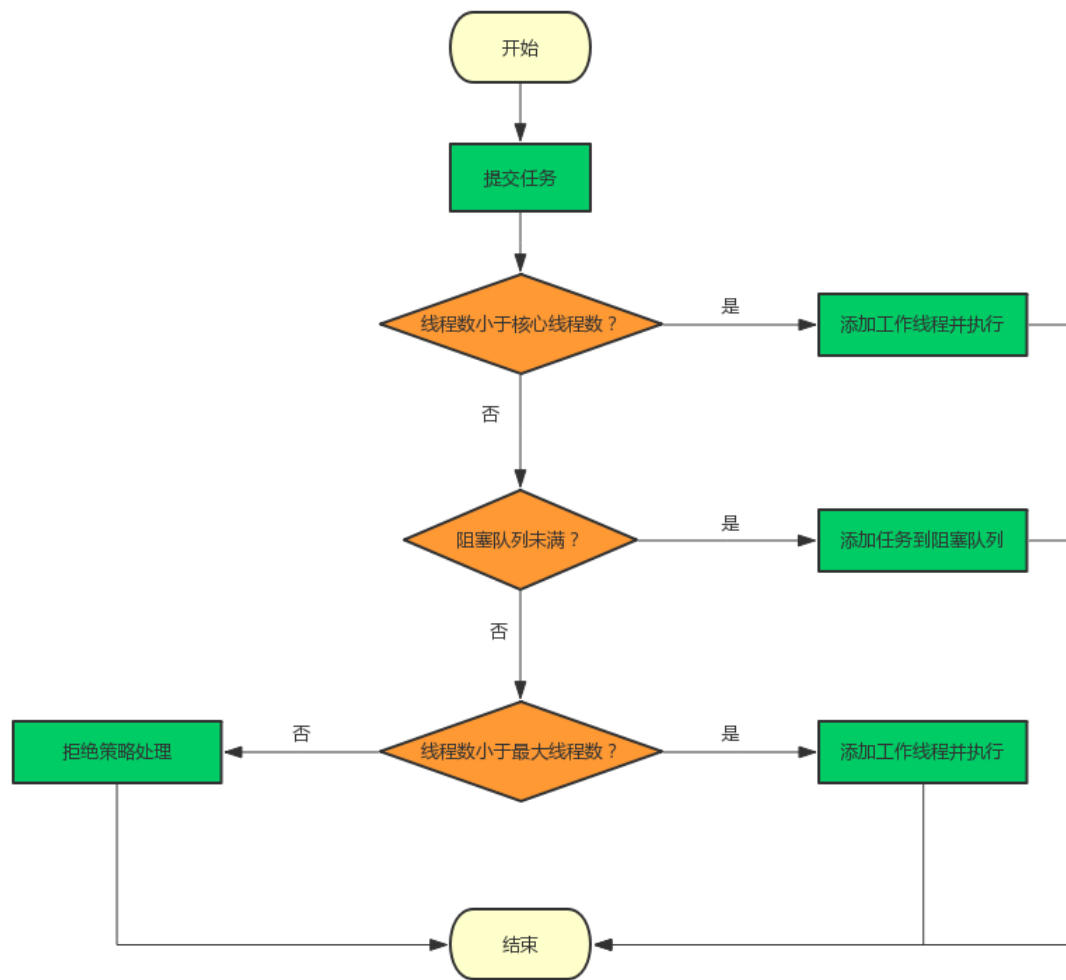
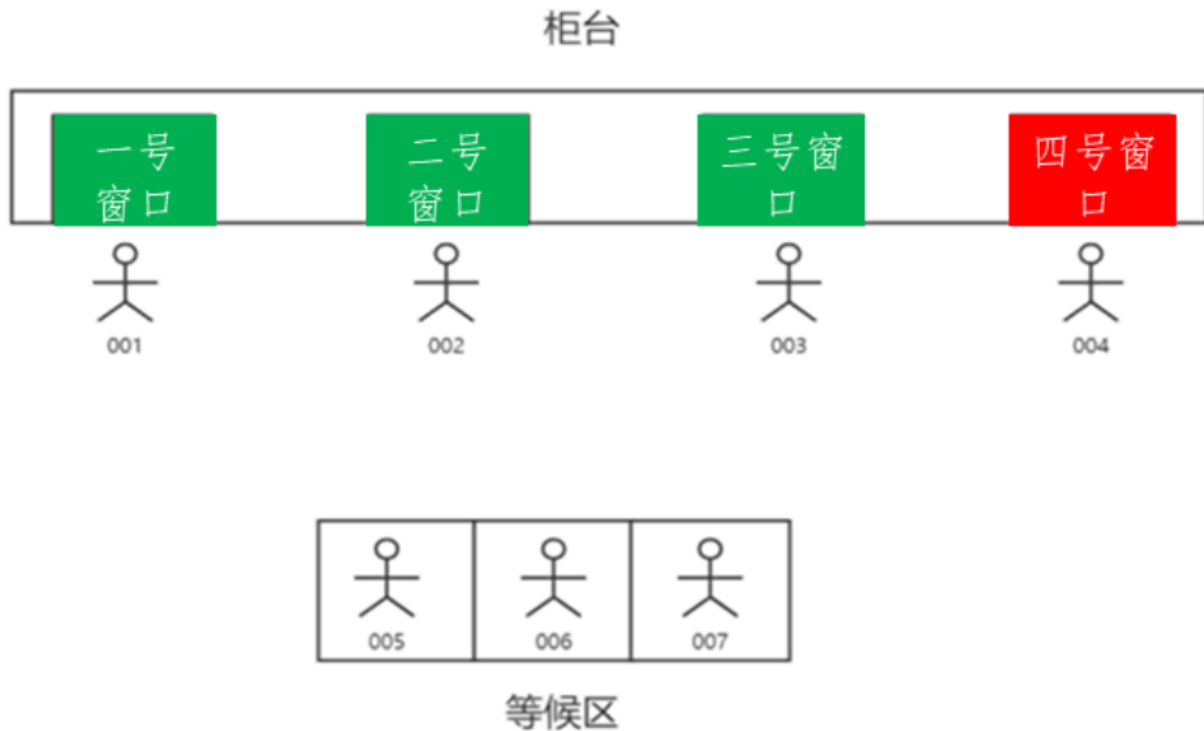


提前在线程池中创建线程，任务来时线程执行任务，
执行完毕后线程休息，等待下一个任务

类比：包工头 提前招募若干工人，有活时安排工人工作，
工人完成工作后休息，等待下一次被安排工作



线程池



• 使用线程池的优点:

- 降低资源消耗: 通过重复利用已创建的线程降低线程创建和销毁造成的消耗。
- 提高响应速度: 当任务到达时, 任务可以不需要等到线程创建就能立即执行。
- 提高线程的可管理性: 线程是稀缺资源, 如果无限制的创建, 不仅会消耗系统资源, 还会降低系统的稳定性, 使用线程池可以进行统一的分配、调度和监控。

线程池

• ThreadPoolExecutor 线程池主要参数:

➤ corePoolSize:

线程池中所保存的**核心**线程数

➤ maximumPoolSize:

线程池中允许的**最大**线程数

➤ keepAliveTime:

线程池中的空闲线程所能持续的**最长时间**

➤ workQueue:

阻塞队列

➤ threadFactory:

创建线程的工厂，主要定义线程名

➤ handler:

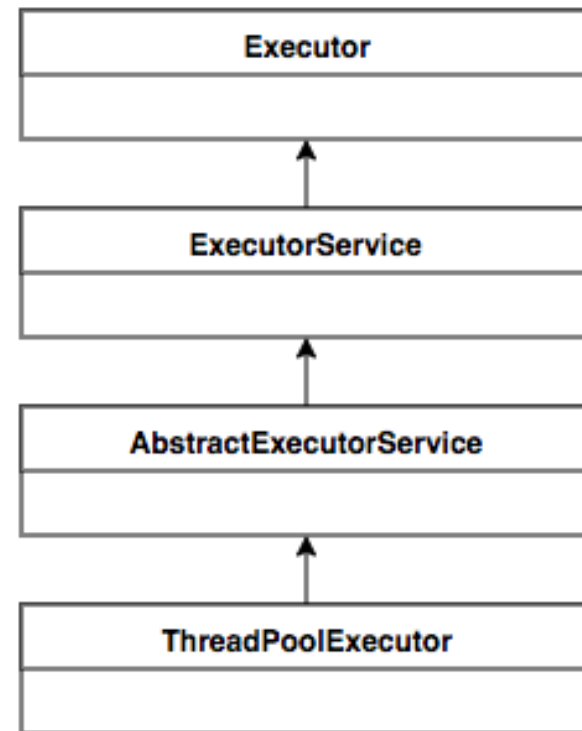
拒绝策略

```
public ThreadPoolExecutor(  
    int corePoolSize,  
    int maximumPoolSize,  
    long keepAliveTime,  
    TimeUnit unit,  
    BlockingQueue<Runnable> workQueue,  
    ThreadFactory threadFactory,  
    RejectedExecutionHandler handler) {  
}
```

线程池

• 线程池的继承关系

- **Executor**: 是一个接口，将业务逻辑提交与任务执行进行分离
- **ExecutorService**: 接口继承了Execute，做了shutdown()等业务的扩展，可以说是**真正的线程池接口**
- **AbstractExecutirService**: 抽象类实现了ExecutorService接口的大部分方法
- **ThreadPoolExecutor**: **线程池的核心实现类**，用来执行被提交的任务





多线程应用 生产者-消费者设计模式

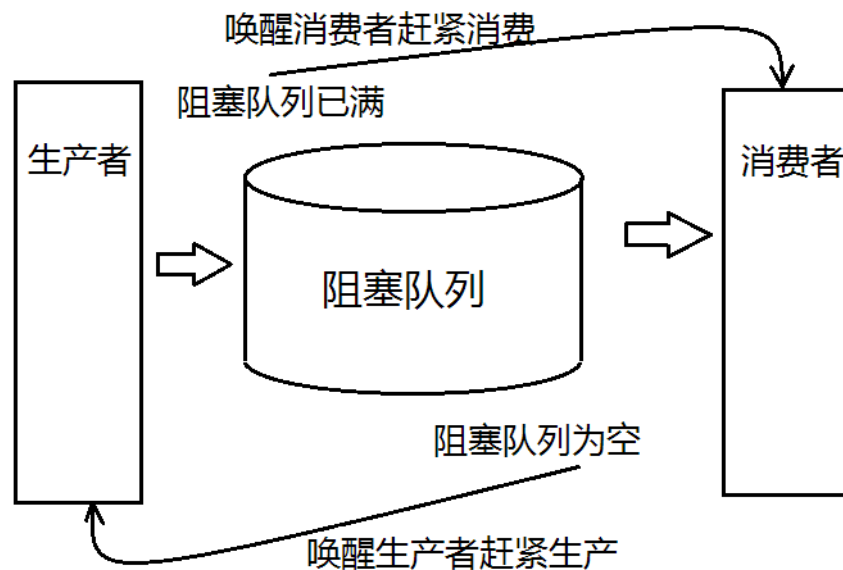
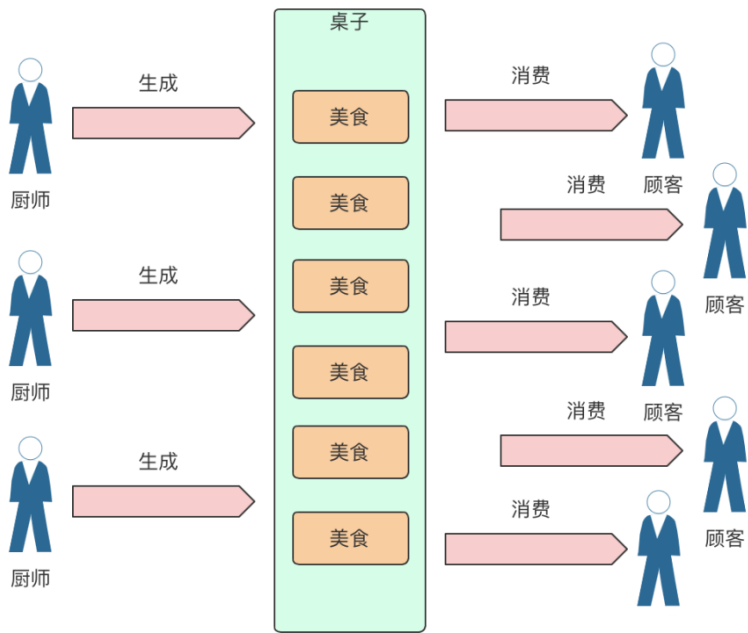


生产者-消费者设计模式

- 在实际的软件开发过程中，经常会碰到如下场景：

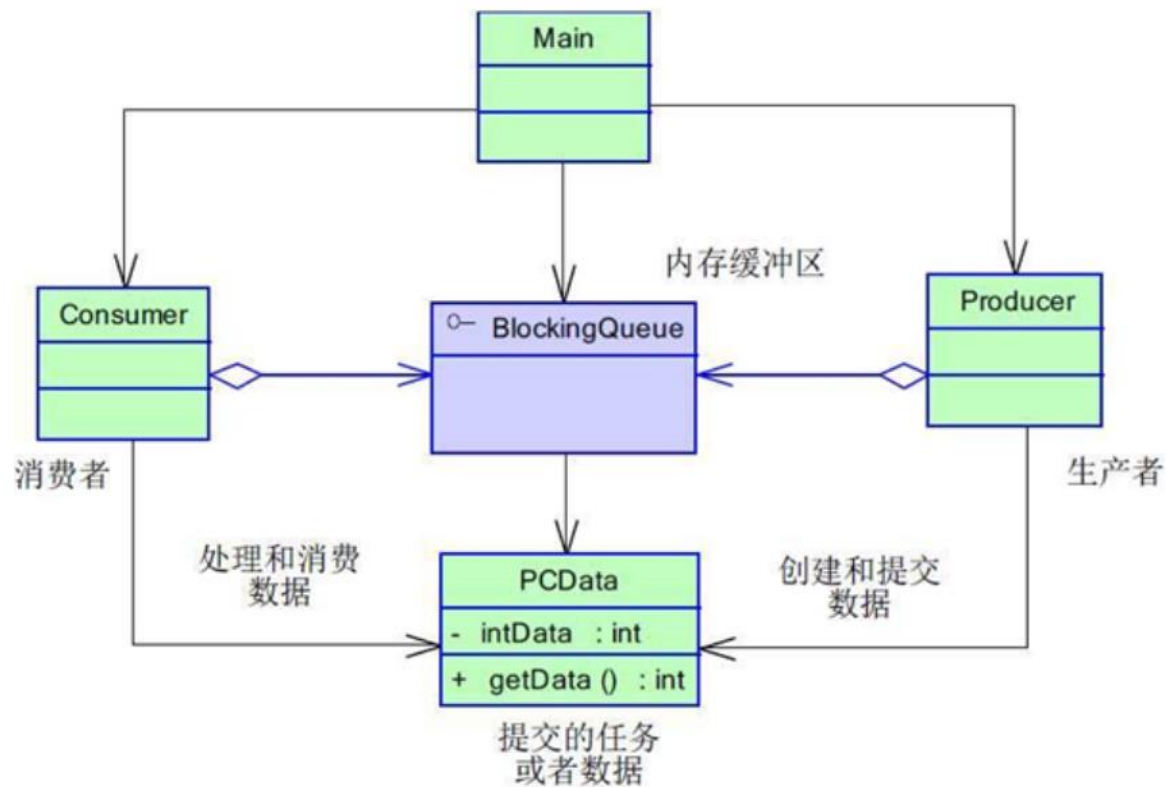
某个模块负责产生数据，这些数据由另一个模块来负责处理（此处的模块是广义的，可以是类、函数、线程、进程等）。

- 产生数据的模块，就形象地称为生产者；而处理数据的模块，就称为消费者。
- 如果生产者生产速度过快，消费者消费的很慢，并且缓存区达到了最大时，缓存区会阻塞生产者，让生产者停止生产，等待消费者消费了数据后，再唤醒生产者。
- 当消费者消费速度过快时，缓存区为空时，缓存区则会阻塞消费者，待生产者向队列添加数据后，再唤醒消费者。





生产者-消费者设计模式



- 好处：
 - 并发（异步）：生产者和消费者各司其职，生产者和消费者都只需要关心缓冲区，不需要互相关注，通过异步的方式支持高并发，将一个耗时的流程拆成生产和消费两个阶段。
 - 解耦：生产者和消费者进行解耦（通过缓冲区通讯）。



生产者-消费者设计模式

- 问题:

- 如何保证缓存区中数据状态的一致性?
- 如何保证消费者和生产者之间的同步和协作关系?

- 方案:

- 加同步锁 (`synchronized`)
- 利用线程内部之间的通信 (`Object`的`wait()` / `notify()`方法)



生产者-消费者设计模式

Java代码实现缓冲区

```
public class Buffer {  
    private List<Integer> data = new ArrayList<>();  
    private static final int MAX = 2;  
    private static final int MIN = 0;  
    public void put(int value){  
        while (true){  
            try { //模拟生产数据  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            synchronized (this){  
                //当容器满的时候, producer处于等待状态  
                while (data.size() == MAX){  
                    System.out.println("buffer is full,waiting ....");  
                    try {  
                        wait();  
                    }  
                }  
            }  
        }  
    }  
}
```

```
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
}  
//没有满, 则继续produce  
System.out.println("producer--"+  
Thread.currentThread().getName()+"--put:" + value);  
data.add(value);  
//唤醒其他所有处于wait()的线程, 包括消费者和生产者  
notifyAll();  
}  
}  
}
```




生产者-消费者设计模式

Java代码实现缓冲区

```
public Integer take(){
    Integer val = 0;
    while (true){
        try { //模拟消费数据
            Thread.sleep(500);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        synchronized (this){
            //如果容器中没有数据，consumer处于等待状态
            while (data.size() == MIN){
                System.out.println("buffer is empty,waiting ...");
                try {
                    wait();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

```
//如果有数据，继续consume
    val = data.remove(0);
    System.out.println("consumer--"+
        Thread.currentThread().getName()+"--take:" + val);
```

//唤醒其他所有处于wait()的线程，包括消费者和生产者

```
        notifyAll();
    }
}
}
```



生产者-消费者设计模式

Java代码实现
生产者

```
public class Producer implements Runnable{  
    private Buffer buffer;  
    public Producer(Buffer buffer) {  
        this.buffer = buffer;  
    }  
    @Override  
    public void run() {  
        buffer.put(new Random().nextInt(100));  
    }  
}
```



生产者-消费者设计模式

Java代码实现

消费者

```
public class Consumer implements Runnable{  
    private Buffer buffer;  
    public Consumer(Buffer buffer) {  
        this.buffer = buffer;  
    }  
    @Override  
    public void run() {  
        Integer val = buffer.take();  
    }  
}
```



生产者-消费者设计模式

- 问题:

- 为什么缓冲区的判断条件是 **while(condition)** 而不是 **if(condition)**?
- Java中要求**wait()**方法为什么要放在**同步块**中?

- 答案:

- 防止线程被错误的唤醒，永远在**while**循环里而不是**if**语句下使用**wait**。这样，循环会**在线程睡眠前后都检查wait的条件**，并在条件实际上并未改变的情况下处理唤醒通知。
- 防止出现**Lost Wake-Up**。

